Laboratorio di Calcolo per Fisici

Esercitazione n. 4, 2-3 Novembre

Canale A-Dec 2026-2027, Docenti: G. Artoni, L. Caprini

Lo scopo della quarta esercitazione di laboratorio è di fare pratica con le istruzioni di controllo di flusso `if...(then)...else`; `while...do`; `for...`, scrivendo un programma che localizza gli zeri di una funzione all'interno di un intervallo dato.

Fate il login utilizzando `lcgaXXX` sia come *userid* sia come *password*, dove `XXX` è il codice a tre cifre associato al vostro gruppo, così come indicato nella comunicazione dei gruppi.
In quanto segue, si assume che la vostra **`home`** sia `/home/studenti/lcgaXXX`. L'indirizzo potrebbe essere diverso, ma potete controllarlo immediatamente aprendo il terminale ed utilizzando il comando `pwd`.

► **Attenzione!** A volte, per via di un errore di programmazione, può succedere che un programma entri in un *loop infinito*. Se questo succede, ci sono due modi di interrompere l'esecuzione:

- Se il programma è in esecuzione nel terminale: premere la combinazione di tasti `CTRL+C`.
- Se il programma è in esecuzione in modalità *background*: aprire una nuova shell. Lanciare il programma `top` (non in modalità *background*). Individuare il PID (*Process Identifier*) del programma bloccato. Premere `q` per uscire da `top`. Digitare nella shell il comando: `kill -9 PID` (Enter).

---

1. Aprite una finestra di *terminale*.
2. Nella vostra home create una cartella chiamata `EX4` e posizionatevi all'interno di quest'ultima. Tutti i file, gli script Python e i codici C dovranno essere contenuti all'interno di questa cartella.

► **Prima parte (obbligatoria)**
Scrivete un programma `sinxx.c` che, utilizzando un opportuno ciclo iterativo, calcoli il valore della funzione $f(x) = \frac{\sin(x)}{x}$ in una serie di punti equispaziati lungo l'asse x, nell'intervallo $x \in [-15, 15]$. In particolare

1. il programma deve chiedere in input il numero dei punti N_p in cui calcolare la funzione $f(x) = \frac{\sin(x)}{x}$. Si introduca un controllo nel codice che continui a chiedere in input il numero dei punti se il valore inserito non è maggiore o uguale a 100 ($N_p \geq 100$).
2. Poiché i punti sono equispaziati, il programma deve calcolare la distanza Δx tra un punto e l'altro nell'intervallo $[-15, 15]$. Per fare ciò chiedetevi quale sia la lunghezza di un generico intervallo $[a, b]$: di quanti segmenti di lunghezza Δx è costituito l'intervallo $[a, b]$ quando in esso sono equidistribuiti N_p punti (estremi dell'intervallo inclusi)? Utilizzate un disegno (ad esempio quello qui sotto con $N_p = 10$) per rispondere a queste domande ed estrapolare dal caso particolare l'espressione generica per il calcolo di Δx.

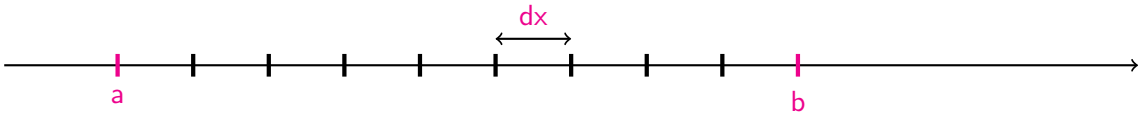


Figura 1: Il generico intervallo $[a, b]$ è suddiviso in $N_p = 10$ punti. Quanto vale Δx?

3. Con un ciclo iterativo, a partire dall'estremo sinistro dell'intervallo, calcolate il valore di $\frac{\sin(x)}{x}$ per ciascuno degli N_p punti nell'intervallo. Si tenga presente che, noto Δx, il punto i-esimo si trova in $x_i = a + i\,\Delta x$, con $i = 0, 1, \ldots, N_p - 1$: il primo punto ($i = 0$) coincide con l'estremo sinistro a, il secondo è in $a + \Delta x$, il terzo in $a + 2\Delta x$ e così via, fino all'ultimo ($i = N_p - 1$) che coincide con b.

4. **Importante:** Se la funzione viene calcolata nel punto $x = 0$ il programma restituisce il valore `nan` (*not a number*), dal momento che si tratta di valutare un'espressione del tipo $\frac{0}{0}$. Il limite per $x \to 0$ di $\frac{\sin(x)}{x}$ è noto e vale $\lim_{x\to 0} \frac{\sin(x)}{x} = 1$. Inserite nel programma un'opportuna istruzione `if` che corregga il risultato numerico con il valore analitico corretto in $x = 0$.

   Attenzione! Non confrontate mai due variabili `double` con l'operatore `==`. Poiché x viene calcolato come `a + i * delta_x`, gli errori di arrotondamento fanno sì che il punto "centrale" valga quasi sempre qualcosa come 10^{-16} invece di 0 esatto, e la condizione `x == 0.0` risulterebbe falsa. Conviene invece verificare che x sia *abbastanza vicino* a zero, cioè che $|x| < \varepsilon$, dove ε è una tolleranza piccola (ad esempio `#define EPSILON 1e-9`). La funzione `fabs` di `math.h` calcola il valore assoluto.

5. Scrivete i risultati su un file `sinxx.dat` di N_p righe e 2 colonne nella forma: `x y`. Per compiere questa operazione potete incollare manualmente l'output dello schermo su un file vuoto o *redirigere* l'output del vostro programma su un file con il comando:

```
./programma > file_out
```

6. Create con python un grafico della funzione $\frac{\sin(x)}{x}$ utilizzando i punti calcolati e salvatelo sul file `sinxx.png`. Quanti zeri ci sono nell'intervallo $[-15, 15]$? Dove sono?

Suggerimento: Una possibile struttura per il nostro codice è:

```c
#include <stdio.h>
#include <math.h>

int main() {
 // definizioni delle variabili necessarie
 ....
 double a = -15.0;
 double b = 15.0; // estremi dell'intervallo. Perche' double?

```

```c
do { // controllo dei dati in lettura
  ...
  /*
  - stampare un messaggio che chiede di inserire Np
  - leggere Np
  - controllare che Np >=100 altrimenti continuare con l'iterazione
  finche' non si verifichi che Np>=100
  */
} while(....);

// calcolo di delta_x
delta_x = ...
// calcolo di sinx/x lungo gli Np punti
for(i = 0; i < Np; i++){
  x = a + ... * delta_x; // calcolo del punto i-esimo
  if(...) { // se x e' (circa) 0 allora fx = 1
    ...
  }
  else { // altrimenti fx = sin(x) / x
    ...
  }
  ...
  // stampa x e fx
 }
}

```

Listato 1: Schema di `sinxx.c`

► **Seconda parte (obbligatoria)**

Scrivete un programma `zero.c` che calcoli automaticamente il numero degli zeri della funzione $f(x) = \frac{\sin(x)}{x}$ nell'intervallo $[a, b] = [-15, 15]$ mediante un algoritmo basato sul teorema degli zeri di una funzione continua: se una funzione $f(x)$, continua in $[a, b]$, è tale per cui $f(a) \cdot f(b) \leq 0$, allora f ammette almeno uno zero in $[a, b]$, cioè un punto x_0 tale che $f(x_0) = 0$.

Nota Bene: se $f(a) \cdot f(b) > 0$ non è detto che f non abbia zeri in $[a, b]$!

L'algoritmo è costituito dai seguenti passaggi:

1. L'intervallo di partenza, $(b - a)$, viene diviso in $N = N_p - 1$ intervalli contigui di larghezza $\Delta x = (b - a)/N$ e di estremi x_L^i, x_R^i, $i = 0, \cdots, (N - 1)$ (si veda la Fig. 2 per chiarezza).

2. Per ciascun intervallo l'algoritmo controlla la presenza di eventuali zeri calcolando il valore di $s = f(x_L^i) \cdot f(x_R^i)$. Se $s < 0$ c'è almeno uno zero, quindi viene incrementato un contatore che tiene in memoria gli zeri calcolati fino a quel momento. Se $s \geq 0$ il contatore non viene incrementato (usiamo la disuguaglianza stretta perché uno zero che cadesse esattamente su un estremo comune a due intervalli verrebbe altrimenti contato due volte).

3. Alla fine dell'esecuzione, il programma restituisce il numero totale di zeri trovati dall'algoritmo, N_{zeri}.

4. **Attenzione:** anche in questo caso è necessario controllare se x_L^i o x_R^i siano (abbastanza vicini a) zero, e in caso porre il valore della funzione calcolata in quel punto pari a 1. Valgono le stesse considerazioni sul confronto tra `double` fatte nella prima parte.

Nel programma il numero di punti deve essere definito attraverso una direttiva al preprocessore in fase di compilazione (quindi non vanno eseguiti controlli sul numero di punti inseriti). Vi ricordo che per definire una costante simbolica in fase di compilazione bisogna compilare con l'opzione `-DNOME_COSTANTE=VALORE_COSTANTE` come nell'esempio seguente:

```
gcc -o zero zero.c -lm -DNP=100
```

dove `NP` è il nome della costante simbolica (*macro*) che conterrà il numero di punti.

Suggerimento: Questa seconda parte è molto simile dal punto di vista del ciclo iterativo. Raddoppiano le variabili da definire perché mentre al posto di x e fx adesso servono x_L, x_R, fx_L e fx_R. Un'ulteriore variabile è il contatore degli zeri `nzeri` che viene incrementato ogni volta che viene trovato uno zero. Ricordatevi di inizializzare il contatore a 0! Il codice avrà la seguente struttura:

```c
#include <stdio.h>
#include <math.h>

int main() {
 // definizioni delle variabili necessarie
 ...
 double a = -15.0;
 double b = 15.0; // estremi dell'intervallo

 int N = NP - 1;
 delta_x = ...;  // calcolo di delta_x
 // calcolo di sinx/x lungo gli Np punti
 for(i = 0; i < N; i++) {
   xL = a + ... * delta_x; // calcolo dell'i-esimo estremo sinistro
   xR = xL + delta_x; // calcolo dell'i-esimo estremo destro
   if(...) { // se xL e' (circa) 0 allora fxL = 1
     ...
   }
   else {
     fxL = sin(xL) / xL;
   }
   if(...) { // se xR e' (circa) 0 allora fxR = 1
     ...
   }
   else {
     fxR = ...
   }
```

```
    if(...) { // se fxL * fxR < 0.0 -> uno zero nell'intervallo
      ... // incremento il contatore degli zeri
    }
  }
  //fuori dal ciclo for stampare il  N e il numero degli zeri trovati
}

```

Listato 2: Schema di `zero.c`

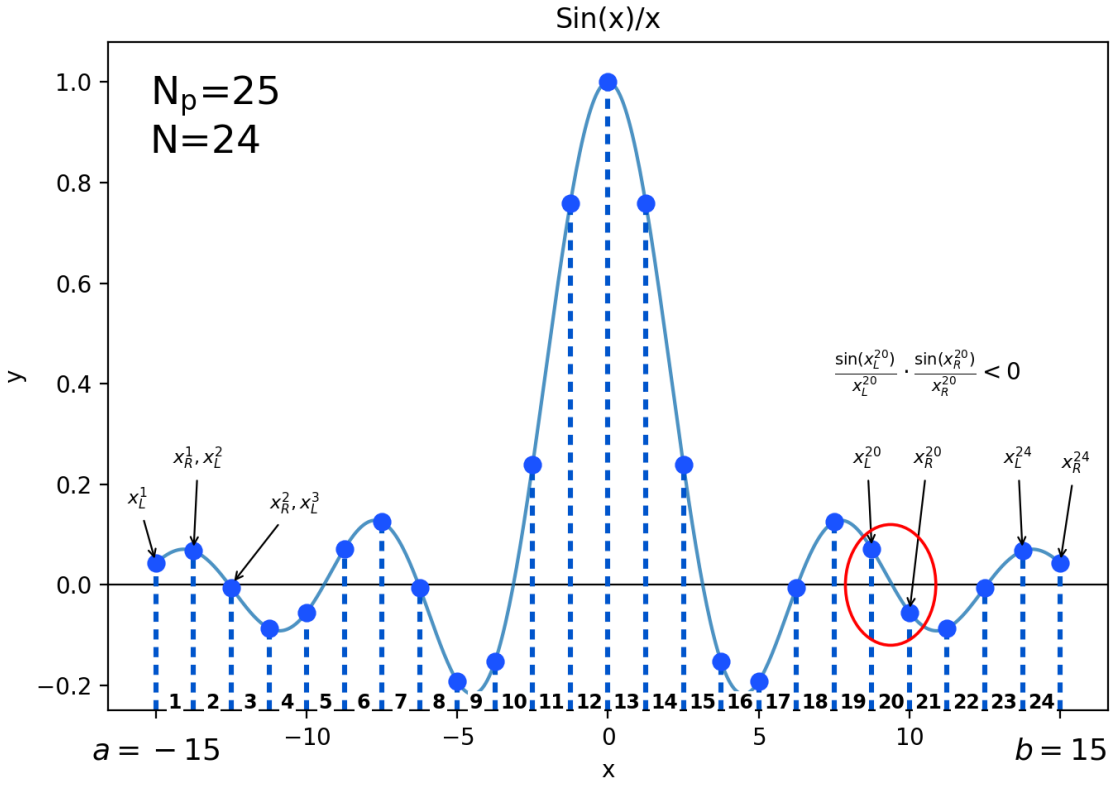


Figura 2: Suddivisione in N intervalli della funzione $f(x) = \sin(x)/x$. Ogni intervallo ha la stessa larghezza. L'estremo destro di ciascun intervallo coincide con l'estremo sinistro dell'intervallo successivo (è vero ovunque tranne che per l'estremo più a sinistra e a destra del grafico). Il cerchio rosso denota una zona dove $f(x_L^i) \cdot f(x_R^i) < 0$, ossia nell'intervallo considerato la funzione interseca l'asse x (in altri termini, c'è uno zero della funzione).

▶ Terza parte (facoltativa)

1. Fate girare il programma `zero.c` con un numero variabile di intervalli $N \geq 1$ (e quindi cambiando in fase di compilazione il valore della costante `NP`, ricordando che `N = Np - 1`), creando un file che contenga due colonne: N e N_{zeri}.

2. Graficate l'andamento di N_{zeri} in funzione di N: qual è il valore minimo di N per cui vengono trovati tutti gli zeri?

3. Se i valori $N = 3, 4, 5, 6$ non fanno parte della vostra scelta iniziale, fate girare il programma anche per questi valori. Quanti zeri trovate? Perché? Aiutatevi graficando i dati con python.

4. Se in un intervallo $[x_L^i, x_R^i]$ è presente uno zero, il valore di x per cui $\sin(x)/x = 0$ si può stimare come $\approx (x_L^i + x_R^i)/2$, con un errore associato $(x_R^i - x_L^i)/2$. Utilizzando diversi valori di N (tutti abbastanza grandi da trovare tutti gli zeri), scrivete un programma che stampi la migliore stima dei valori di x per cui $\sin(x)/x = 0$ (ed errore associato).